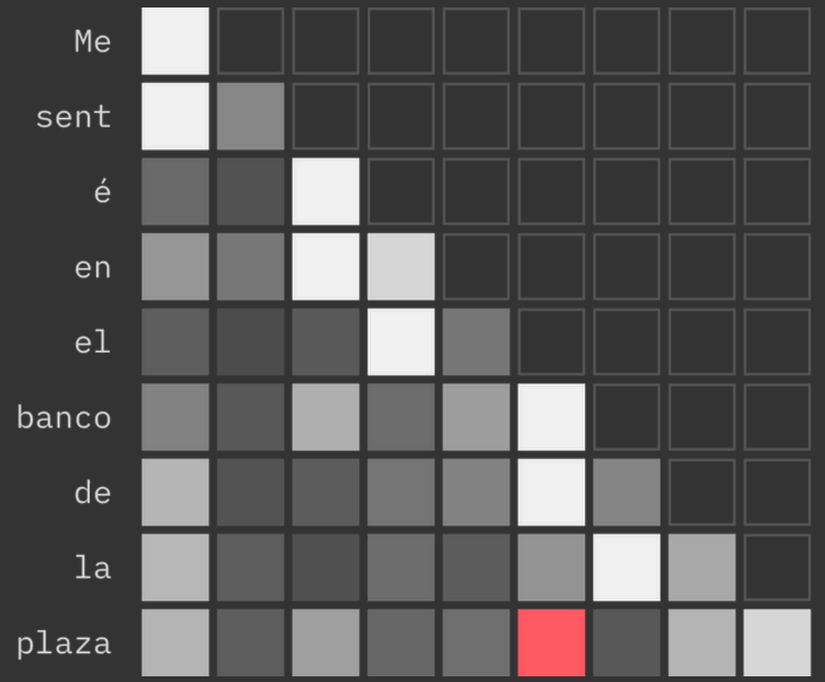
# Clase 5
# Transformers

## [CC66Q] Procesamiento de Lenguaje Natural

Andrés Abeliuk y Fabrizzio Pezzolla

Diplomado en Inteligencia Artificial






# Clase de hoy

**Parte 1** **Atención**

Masked Self-Attention en Transformers

**Parte 2** **Transformers**

Encoder-Decoder, Encoder-only (BERT), Decoder-only (GPT)

**Break**

**Notebook** GPT. BERT.

**Demo** BERTopic

# Translation



**2014**

- Encoder–decoder.
- Toda la oración debe caber en el vector de estado oculto h, y falla con oraciones largas.

**2015**

- Idea: Una red neuronal que considera los estados ocultos del encoder en cada paso e informa al decoder. Se le denomina mecanismo de alineamiento (atención).

**2016**

- **Google Translate** lleva la idea a producción, con LSTM de 8 capas por lado.

**Problema:** Entrenar es **muy** lento. Una RNN tiene que ver las palabras en orden.

Bahdanau et al. (2015); Wu et al. (2016); Vaswani et al. (2017)

Parte 1

# Atención

Masked Self-Attention en Transformers

# Semántica vectorial

## ¿Qué hemos conversado?

**Semántica vectorial**

- Uso como significado.
- Contexto como significado.
- Hipótesis distribucional.

**Representaciones vectoriales**

- Métodos de conteo (BoW, TF-IDF, LSA)
- Representaciones aprendidas (Word2Vec)

**Composicionalidad**

- El perro muerde al hombre; el hombre muerde al perro.

# Semántica vectorial

## Dependencia en el contexto

Adaptado de Jurafsky y Martin

# Semántica vectorial

## Dependencia en el contexto

- La **gallina** no cruzó la calle porque estaba demasiado **cansada**.
- La gallina no cruzó la **calle** porque estaba demasiado **transitada**.

Adaptado de Jurafsky y Martin

# Semántica vectorial

## Dependencia en el contexto

- La **gallina** no cruzó la calle porque estaba demasiado **cansada**.
- La gallina no cruzó la **calle** porque estaba demasiado **transitada**.

- **Las llaves** del portón **están** en la mesa.

Adaptado de Jurafsky y Martin

# Semántica vectorial

## Dependencia en el contexto

- La **gallina** no cruzó la calle porque estaba demasiado **cansada**.
- La gallina no cruzó la **calle** porque estaba demasiado **transitada**.

- **Las llaves** del portón **están** en la mesa.

- Caminaba cerca de la **montaña** y noté las flores que crecían en su **falda**.

Adaptado de Jurafsky y Martin

# Semántica vectorial

## Dependencia en el contexto




Adaptado de Jurafsky y Martin

# Atención

La **atención** es el mecanismo en el transformer que pondera y combina las representaciones de **otros tokens** pertinentes en el contexto de la capa *k* para construir las representaciones de la capa *k+1*.

Una cabeza de atención está dividida en tres roles:

- **query**: El elemento que se compara con los demás inputs.
- **key:** Los otros inputs que se comparan con el elemento actual.
- **value**: El valor de los otros elementos que modifica el output del elemento actual.

$$\mathbf{q}_i = \mathbf{x}_i W^Q \qquad \mathbf{k}_i = \mathbf{x}_i W^K \qquad \mathbf{v}_i = \mathbf{x}_i W^V$$

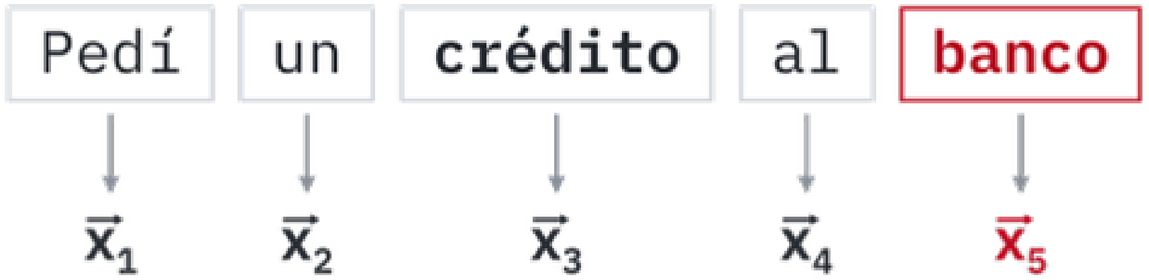
Jurafsky y Martin

# Atención



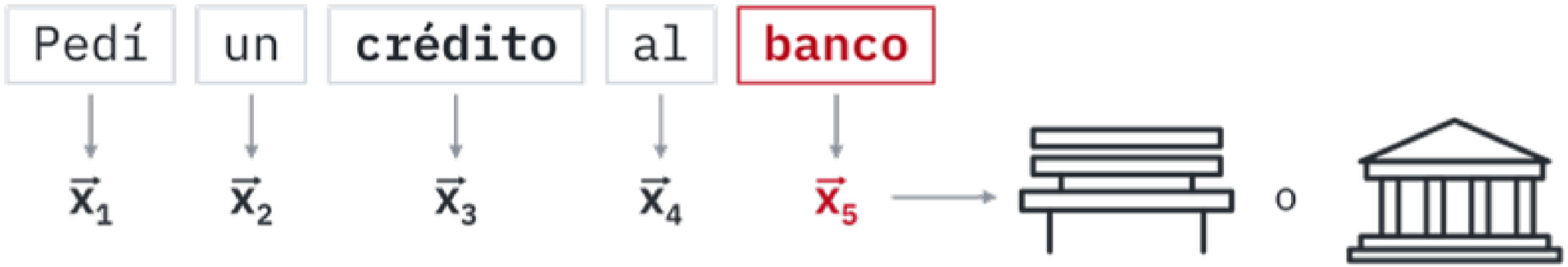

Adaptado de 3Blue1Brown

# Atención



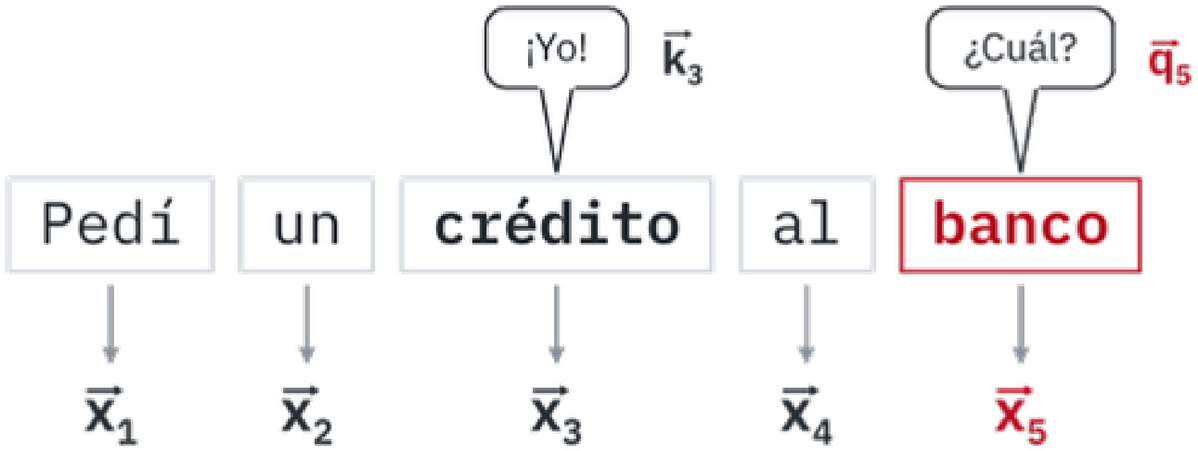

Adaptado de 3Blue1Brown

# Atención




Adaptado de 3Blue1Brown

# Atención




Adaptado de 3Blue1Brown

# Atención




Adaptado de 3Blue1Brown

# Atención




Adaptado de 3Blue1Brown

# Atención



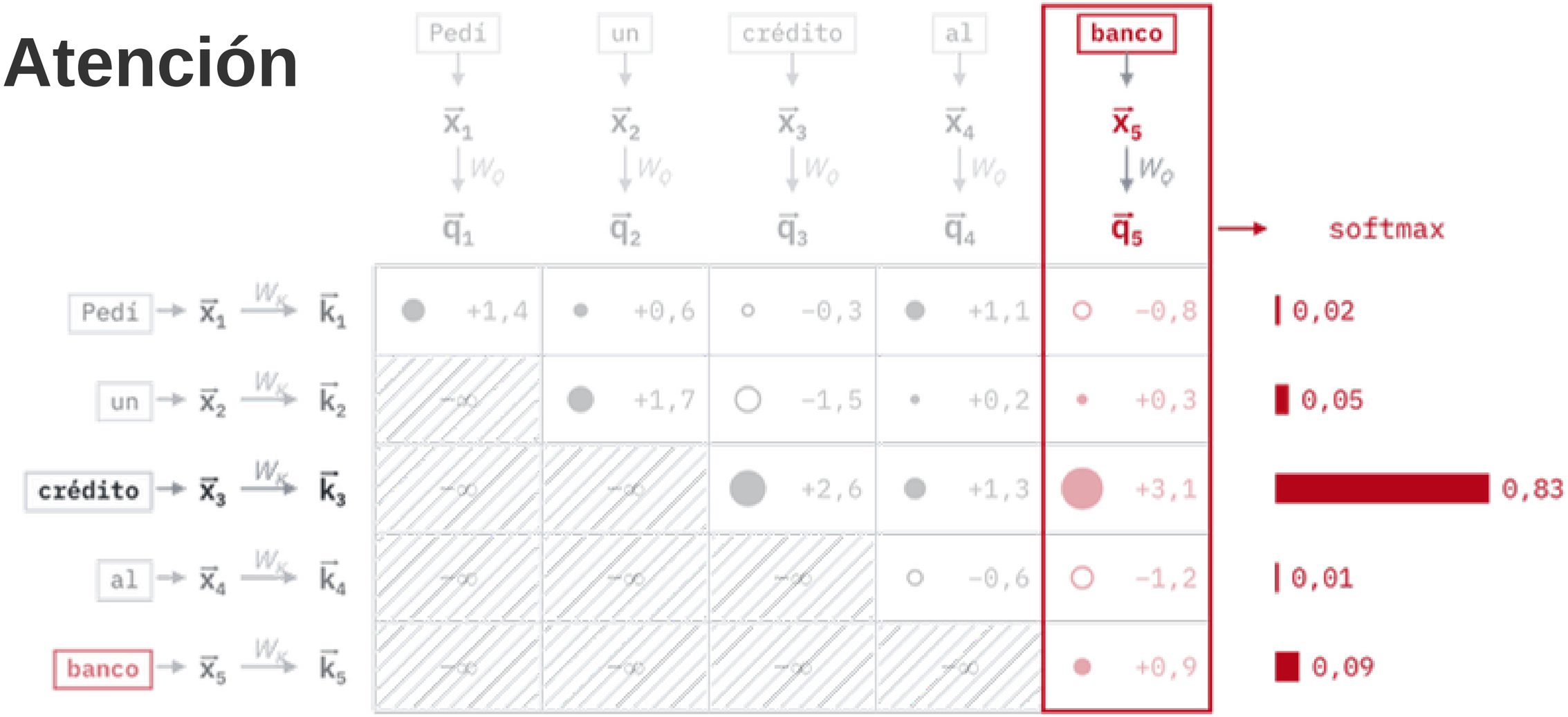

Adaptado de 3Blue1Brown

# Atención



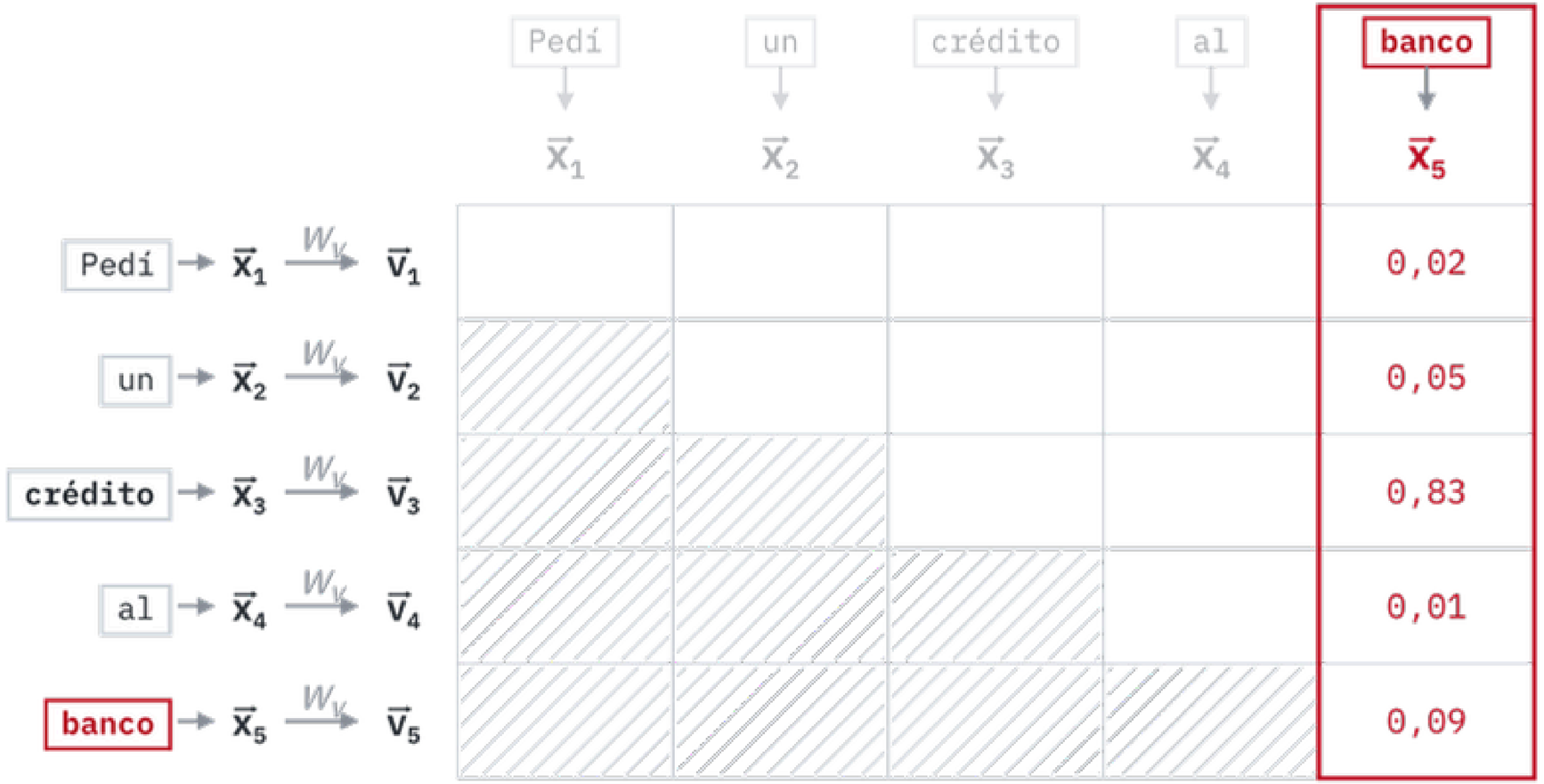

Adaptado de 3Blue1Brown

# Atención



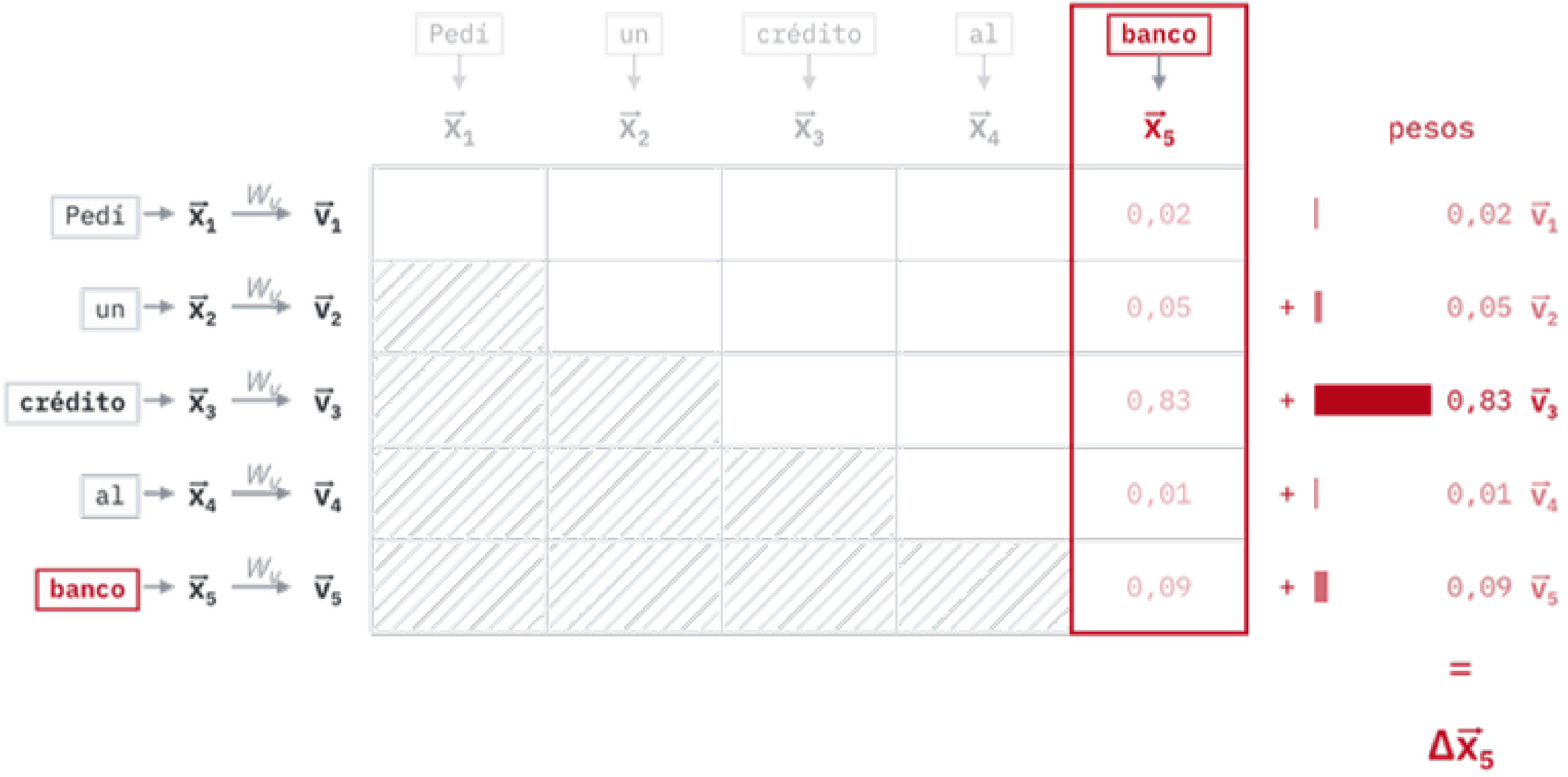

Adaptado de 3Blue1Brown

# Atención



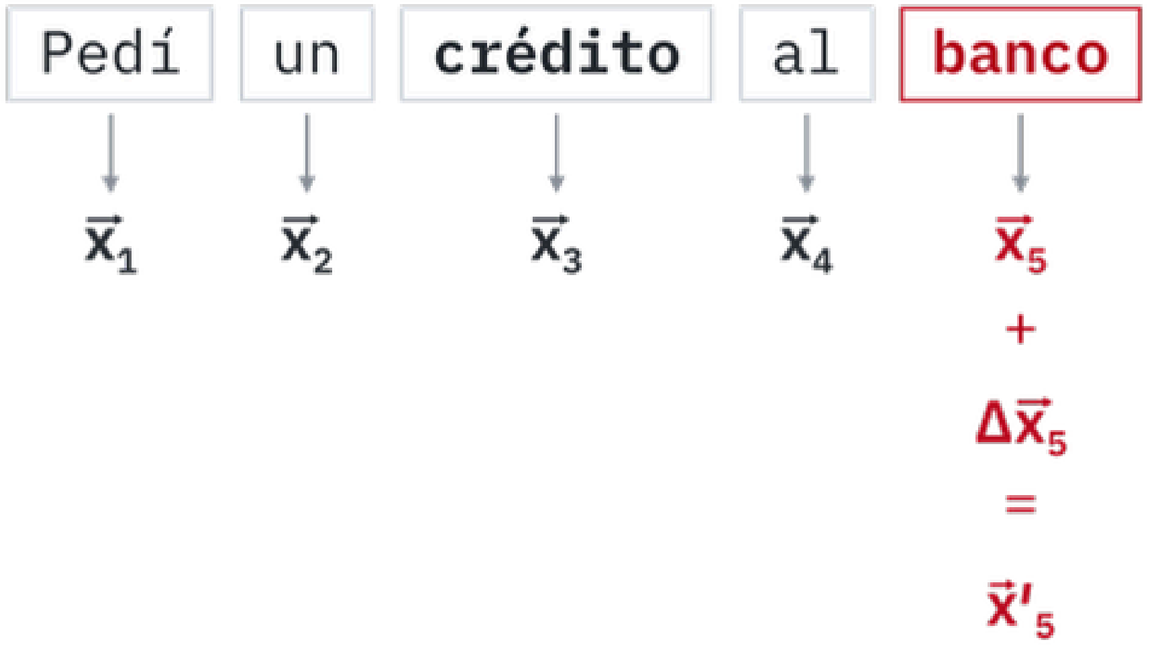

Adaptado de 3Blue1Brown

# Atención



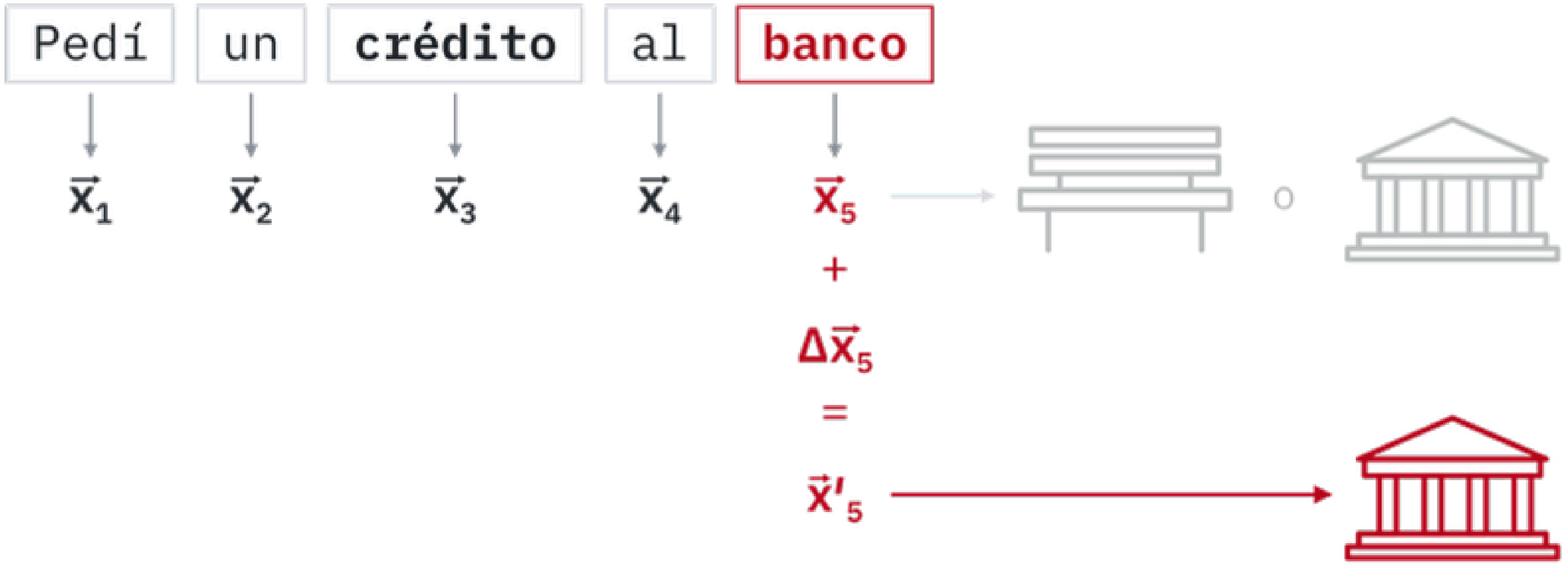

Adaptado de 3Blue1Brown

# Atención




Adaptado de 3Blue1Brown

# Atención




Adaptado de 3Blue1Brown

# Atención

La **atención** es el mecanismo en el transformer que pondera y combina las representaciones de **otros tokens** pertinentes en el contexto de la capa *k* para construir las representaciones de la capa *k+1*.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Jurafsky y Martin; Vaswani et al. (2017)

# Atención

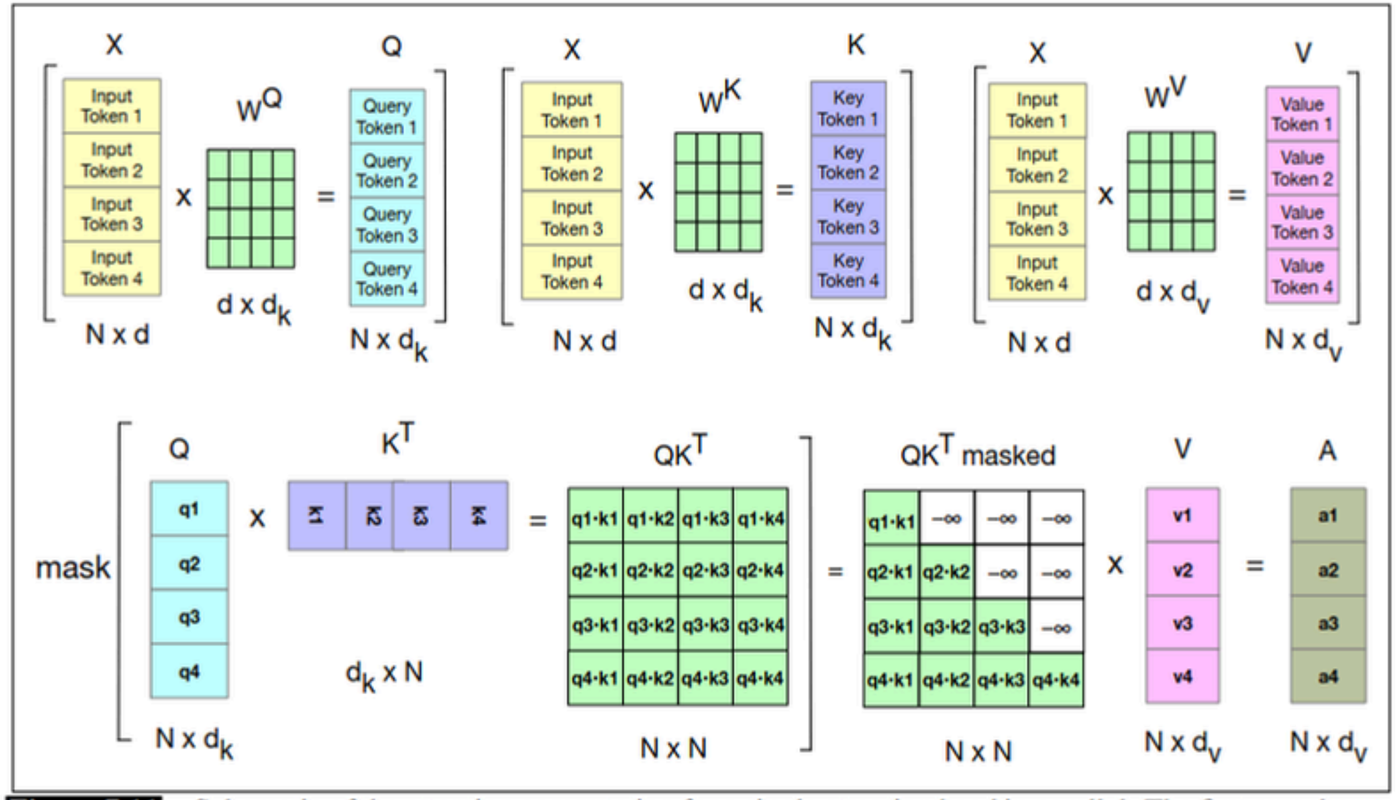



**Figure 7.11** Schematic of the attention computation for a single attention head in parallel. The first row shows the computation of the **Q**, **K**, and **V** matrices. The second row shows the computation of $\mathbf{QK}^T$, the masking (the softmax computation and the normalizing by dimensionality are not shown) and then the weighted sum of the value vectors to get the final attention vectors.

Jurafsky y Martin

Parte 2

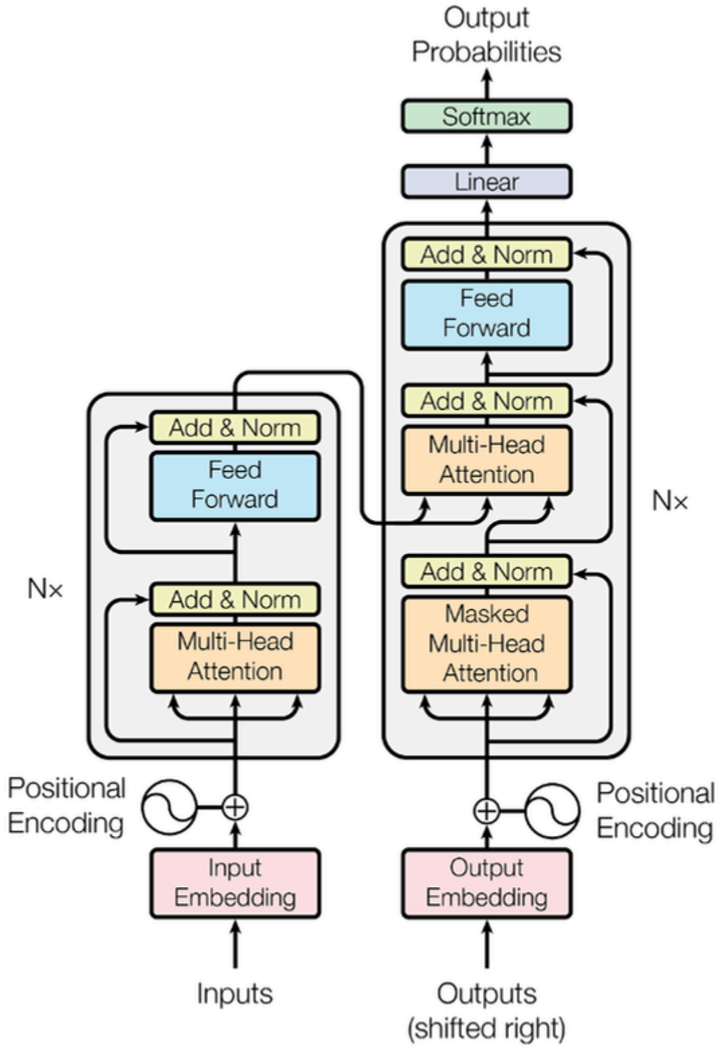
# Transformers

Encoder-Decoder, Encoder-only (BERT), Decoder-only (GPT)

# Transformer

- Propuesto en *Attention is All You Need*.
- Originalmente propuesto para traducción de textos.
- Arquitectura Encoder-Decoder.

GPUs love it!

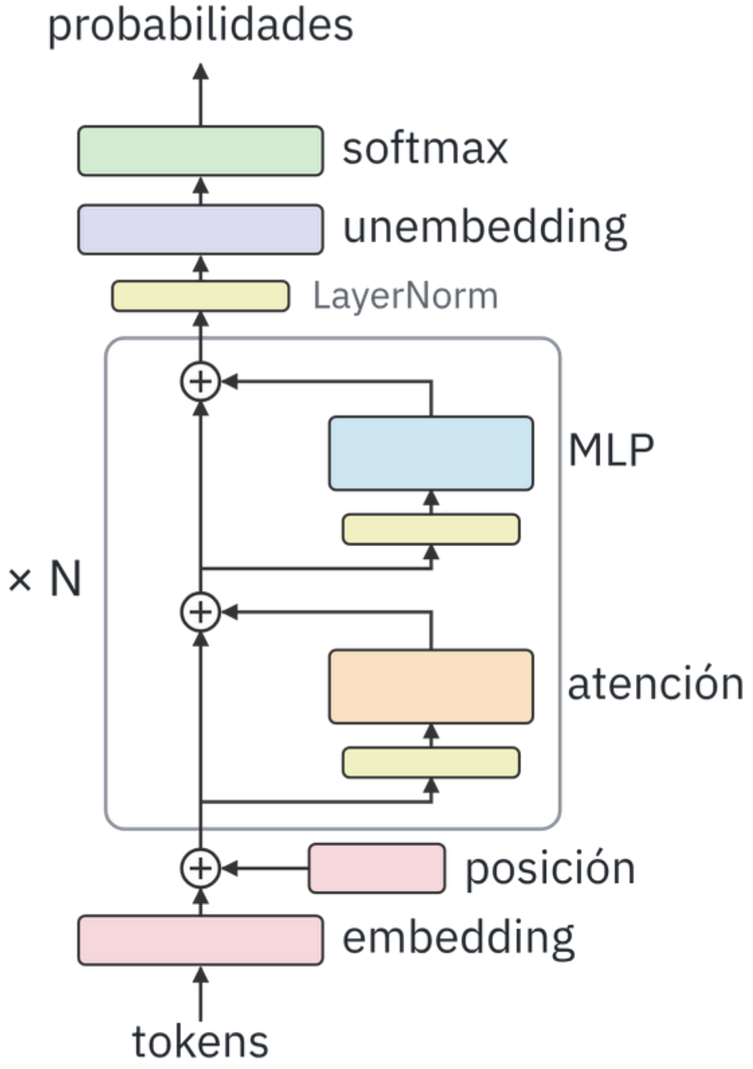



Vaswani et al. (2017)

# Transformer

**Encoder/Decoder**

1. Capa de embedding
2. Capa de posición
3. Capa de atención
4. Capa feed-forward
5. Capa de normalización
6. Capa de unembedding



# Transformer

## Embeddings + Posición

Dos matrices entrenables.

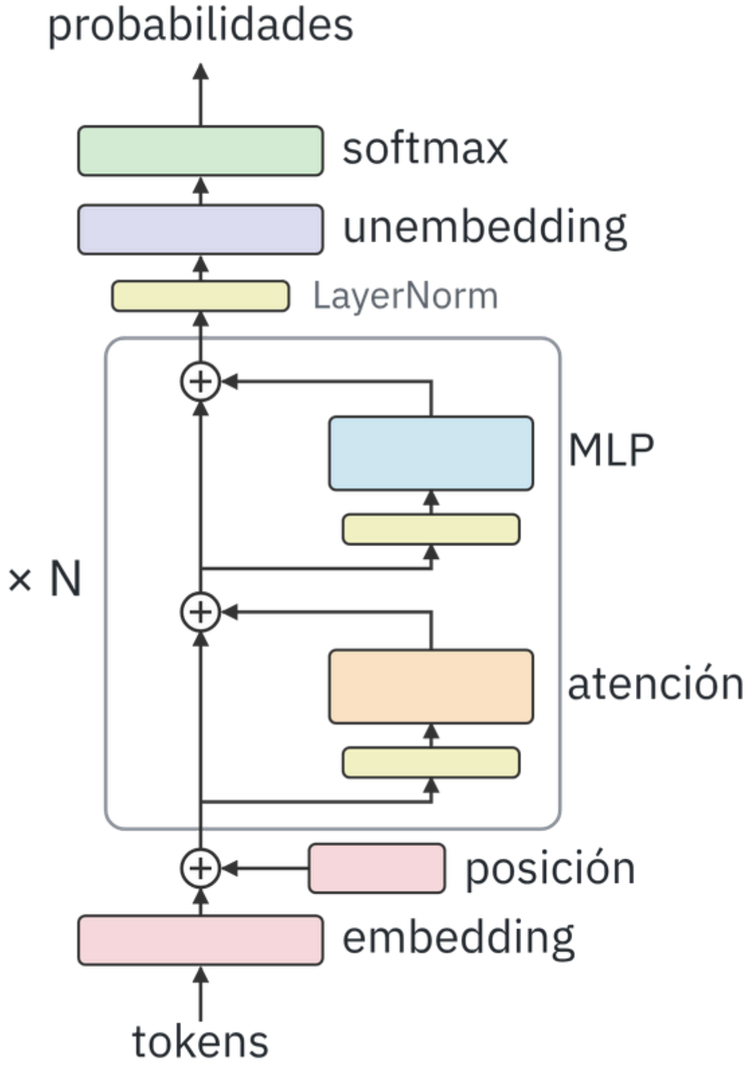
Matriz de embedding

- Una fila por token del vocabulario $E\colon\ |V| \times d$

Matriz  de posición

- Una fila por posición del contexto $P\colon\ N \times d$

$$\vec{x}_i = E[t_i] + P[i]$$



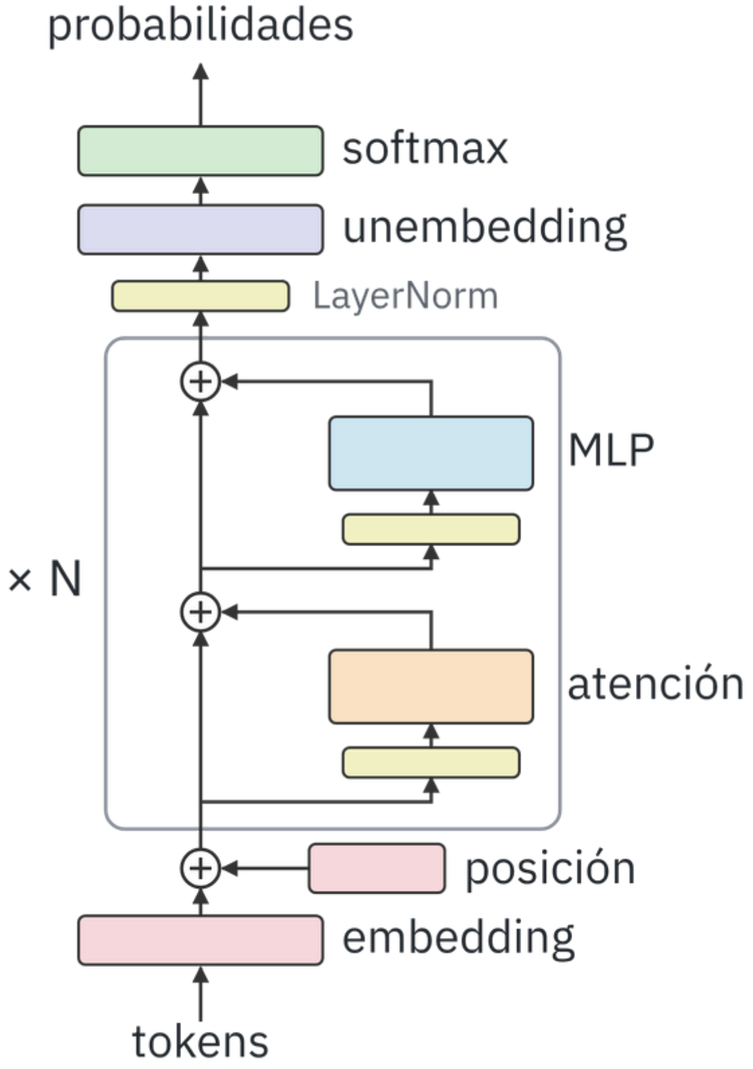
# Transformer

## Atención

- Multi-head attention: varias cabezas en paralelo.
- Cada cabeza tiene sus propias matrices de query, key y value.
- Cada una trabaja con vectores más chicos: d/h números, con h cabezas.
- Sus resultados se juntan y una matriz de salida los vuelve al tamaño d.
- Cada cabeza aprende a mirar otra cosa.

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, W^O$$

$$\mathrm{head}_i = \mathrm{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V)$$




Vaswani et al. (2017)

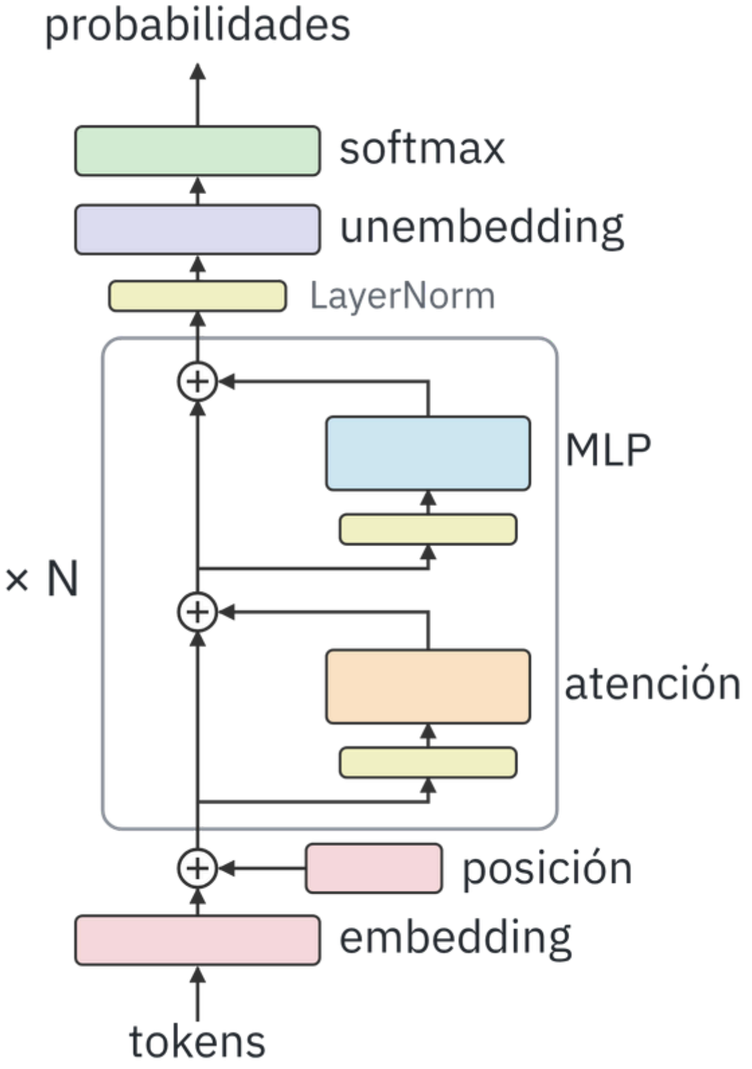
# Transformer

## Feed-Forward

- También conocida como Multi-Layer Perceptron.
- Red de dos capas completamente conectadas.
- Se aplica **por vector**.

- Se multiplica la dimensión de la capa oculta por 4.
- Funciones de activación: ReLU, GLU, SwiGLU, etc.

- En estas capas es donde los transformers “almacenan” información.




Geva et al. (2021)

# Transformer

## Normalización

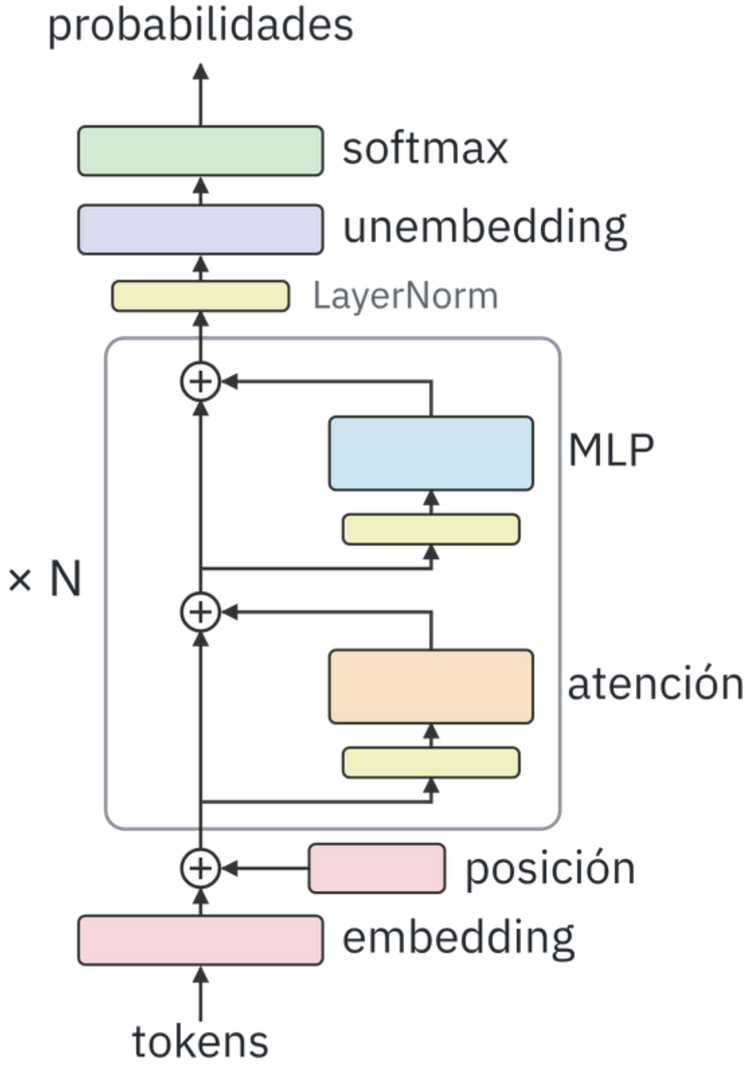
- Se aplica **por vector**.

- Centra el vector y lo escala a desviación estándar 1.
- Dos vectores entrenables: γ y β.

- Variante actual: RMSNorm, que no resta μ.

$$\mathrm{LayerNorm}(\vec{x}) = \vec{\gamma} \odot \frac{\vec{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \vec{\beta}$$

$$\mu = \frac{1}{d}\sum_{j=1}^{d} x_j \qquad \sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2$$



# Transformer

## Capa de unembedding

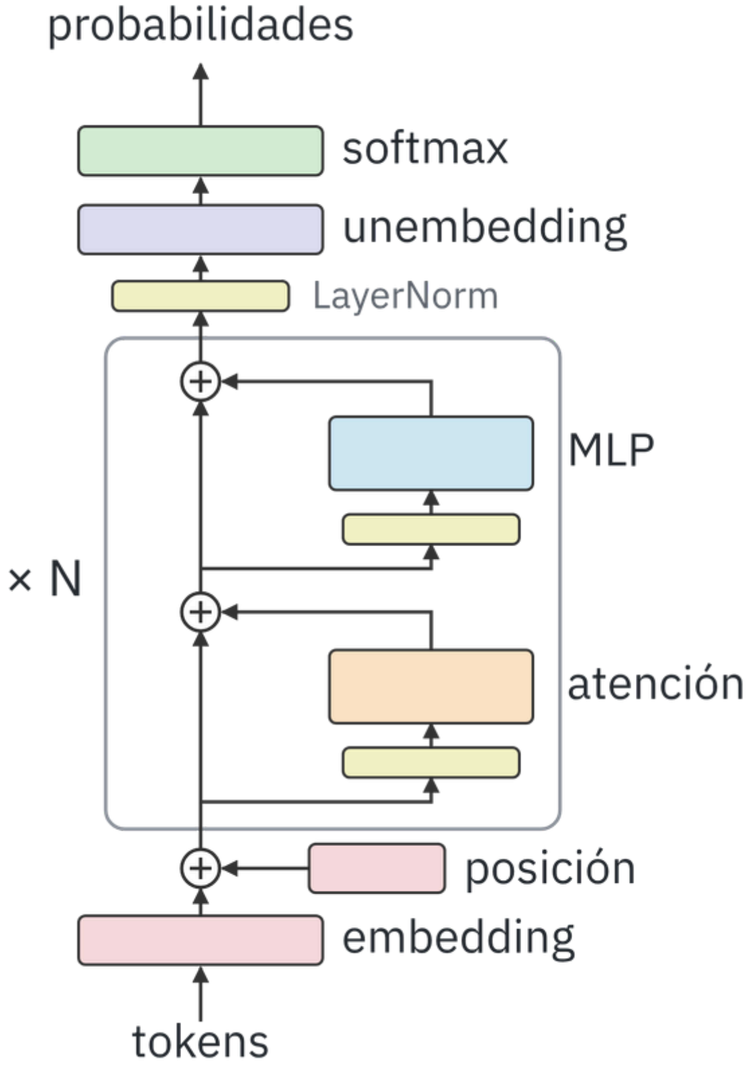
- Se aplica **por vector**.

- Un puntaje (logit) por cada token del vocabulario.
- Softmax convierte los puntajes en probabilidades.

- Muchos modelos usan el embedding, transpuesto.

$$\vec{u} = \vec{x}\, W_U \qquad \vec{p} = \text{softmax}(\vec{u})$$

$$W_U\colon\ d \times |V|$$

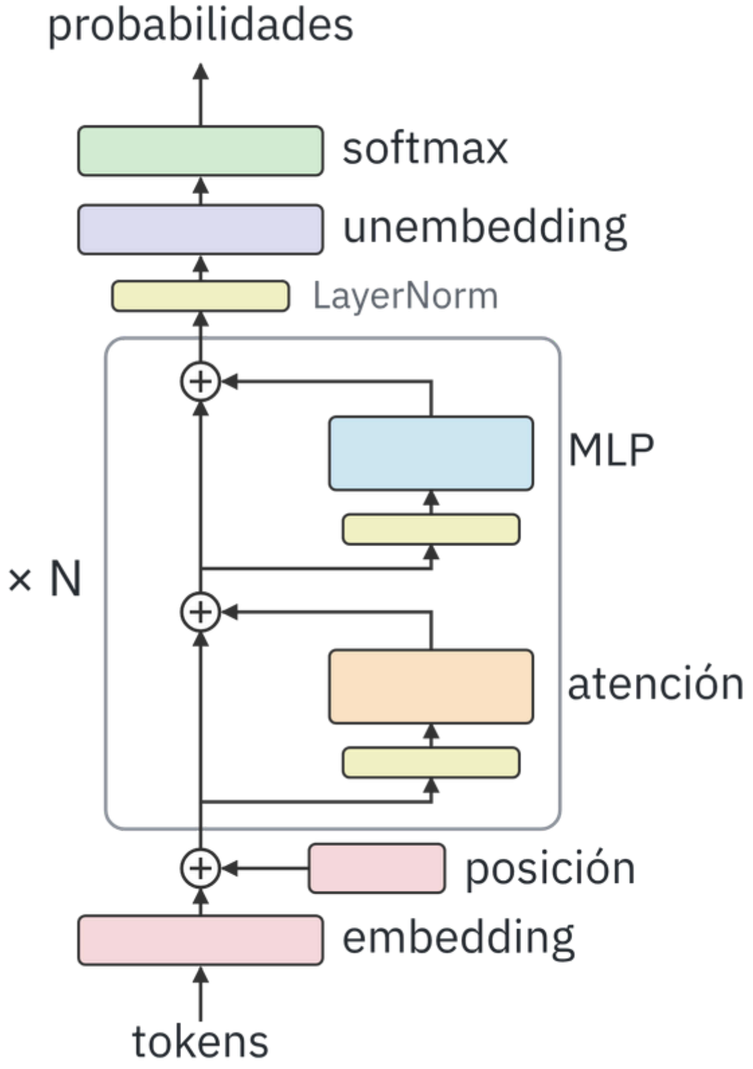


# Transformer

## Softmax

- Al final de todo lo anterior, obtenemos **logits**.
- Para convertir estos logits a probabilidades, usamos la función **softmax**.

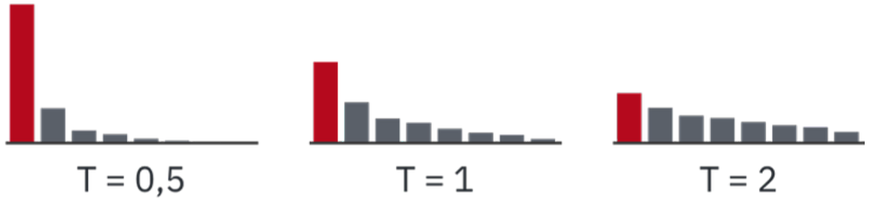
$$p_i = \frac{e^{u_i}}{\sum_{j=1}^{|V|} e^{u_j}}$$



# Transformer

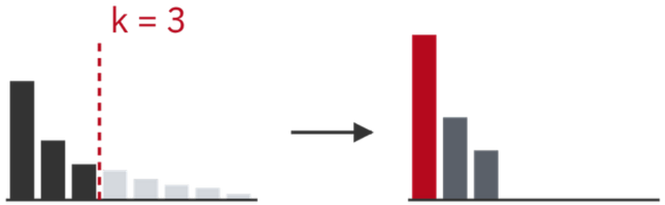
## Random sampling

- Temperatura



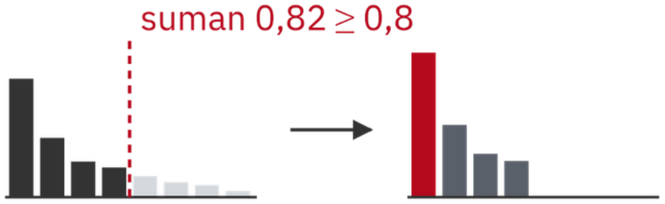

$$p_i = \frac{e^{u_i/T}}{\sum_{j=1}^{|V|} e^{u_j/T}}$$

- Top-k




$$p'_i = \frac{p_i}{\sum_{j \in V^{(k)}} p_j} \quad i \in V^{(k)}$$

- Top-p




$$\sum_{i \in V^{(p)}} p_i \geq p$$

Jurafsky y Martin; Holtzman et al. (2020)

# Referencias


3Blue1Brown (2024), «Attention in transformers, step-by-step»
https://www.3blue1brown.com/lessons/attention

Jurafsky y Martin, Speech and Language Processing, 3.ª ed., capítulo 7
https://web.stanford.edu/~jurafsky/slp3/

Vaswani et al. (2017), «Attention Is All You Need»
https://arxiv.org/abs/1706.03762

Wu et al. (2016), «Google's Neural Machine Translation System»
https://arxiv.org/abs/1609.08144

Bahdanau et al. (2015), «Neural Machine Translation by Jointly Learning to Align and Translate»
https://arxiv.org/abs/1409.0473

Geva et al. (2021), «Transformer Feed-Forward Layers Are Key-Value Memories»
https://arxiv.org/abs/2012.14913

Holtzman et al. (2020), «The Curious Case of Neural Text Degeneration»
https://arxiv.org/abs/1904.09751